

Algoritmi-Modulo algoritmi per bioinformatica

Alberi Binari di Ricerca-Capitolo 12 CLRS

Roberto Posenato

versione 2026-1, 28/03/2026

Sommario

- 1 Introduzione
- 2 Definizione di albero binario di ricerca (ABR)
- 3 Attraversamento
 - Simmetrico (in-order)
 - Pre-ordine e Post-ordine
- 4 Interrogazione
 - Search
 - Minimum, Maximum
 - Successor, Predecessor
- 5 Inserimento e cancellazione
 - Inserimento
 - Cancellazione
- 6 Costruzione ABR
- 7 Esercizi

*“Tell me and I forget, teach me and I remember,
involve me and I learn.”*

Benjamin Franklin

“A student who achieves knowledge through free investigation and spontaneous effort will be able to retain that knowledge and will have acquired a methodology that can serve for a lifetime.”

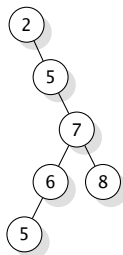
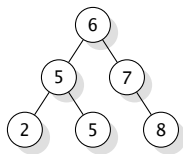
Jean Piaget

Scopo della lezione

- Gli alberi binari di ricerca sono strutture dati che possono essere usate come *dizionari* o *code di priorità*.
- Le operazioni di base sugli alberi binari di ricerca hanno *costo proporzionale all'altezza* dell'albero.
- È importante quindi avere alberi binari di ricerca il più possibile bilanciati (aventi cioè altezza $\approx \lg n$).
- In questa lezione si apprenderà come gestire gli alberi binari di ricerca.
- Quando non è possibile costruire alberi di ricerca bilanciati, si dovrebbero usare delle varianti come gli **alberi rossi-e-neri** perché assicurano un'altezza $O(\lg n)$ nel caso peggiore (non in programma quest'anno).
- Altri tipi di alberi di ricerca sono i **B-alberi**, particolarmente adatti a mantenere grosse quantità di dati nella memoria secondaria (sono in programma nell'insegnamento "Basi di dati").

Definizione di albero binario di ricerca (ABR)

- È un albero binario dove ogni nodo rappresenta un oggetto (informazione da gestire);
- Un nodo contiene: la chiave *key* (che individua i dati satelliti da rappresentare), i riferimenti **left**, **right** e **p** che puntano al figlio sinistro, destro e parente, rispettivamente.
- La radice **root** è l'unico nodo con **p** = NULL.



Esempio due alberi binari che rappresentano lo stesso insieme di dati.

Definizione di albero binario di ricerca

Proprietà fondante

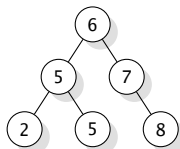
Proprietà fondante

Le chiavi in un albero binario di ricerca (ABR) soddisfano la seguente proprietà:

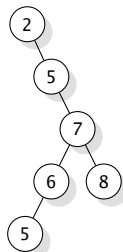
Sia x un nodo.

*Se y è **un** nodo del sottoalbero **sinistro** di x , $y.key < x.key$.*

*Se y è **un** nodo del sottoalbero **destro** di x , $y.key > x.key$.*



ABR bilanciato



ABR non bilanciato

Esempio due alberi binari che rappresentano lo stesso insieme di dati.

Definizione di albero binario di ricerca

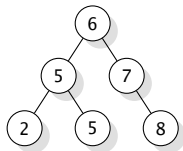
Albero Bilanciato

Sia T un albero binario. In generale, T è detto **bilanciato** quando la sua altezza è $O(\lg n)$, dove n è il numero di nodi.

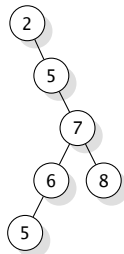
Più specificamente, T è detto **bilanciato in altezza (height-balanced)** se

$$\forall t \in T, \quad |h(t_{sx}) - h(t_{dx})| \leq k$$

dove t_{sx} e t_{dx} sono i sottoalberi sx e dx di t e $h(t)$ è l'altezza del sottoalbero radicato in t . Solitamente $k = 1$.



ABR bilanciato



ABR non bilanciato

Attraversamento

Simmetrico (in-order)

- Algoritmo ricorsivo che sfrutta la proprietà fondante per **visitare le chiavi in modo ordinato**.
- **Idea:** partendo dalla radice, in ordine, si visitano 1) le chiavi del sottoalbero sx ricorsivamente, 2) la chiave della radice e, 3) le chiavi del sottoalbero dx ricorsivamente.

Algoritmo: INORDERTREEWALK(x)

Input: Un nodo x di un ABR

1: **if** $x == null$ **then return**

2: INORDERTREEWALK($x.left$)

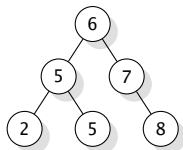
3: VISIT($x.key$)

// VISIT è una procedure esterna

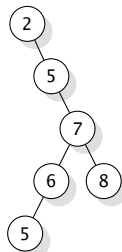
4: INORDERTREEWALK($x.right$)

Attraversamento

Simmetrico (in-order)



ABR bilanciato



ABR non bilanciato

Esempio due alberi binari che rappresentano lo stesso insieme di dati.

- Se `VISIT()` stampa il valore della chiave, l'output di `INORDERTREEWALK` su questi alberi è 2, 5, 5, 6, 7, 8 per entrambi!
- La correttezza dell'algoritmo si dimostra per induzione (lasciata per esercizio).

Teorema 1

Se x è la radice di un ABR di n nodi, e il tempo di esecuzione di una chiamata a VISIT è $O(1)$, l'esecuzione di INORDERTREEWALK richiede tempo $\Theta(n)$.

- $T(n)$ = tempo di esecuzione INORDERTREEWALK con albero di n nodi.
- $T(n) = \Omega(n)$: banale.
- Si deve dimostrare che $T(n) = O(n)$.
- $T(0) = c$ per il check che il nodo è null.
- Sia ora $n > 0$, sotto albero sx di k nodi e sotto albero dx di $n - k - 1$ nodi.
- $T(n) \leq T(k) + T(n - k - 1) + d$ dove d è un limite superiore al tempo costante per eseguire il corpo della chiamata corrente.
- Metodo di sostituzione: si assume che $T(n) \leq (c + d)n + c = O(n)$.
- Per $n = 0$ è banale. Per $n > 0$, $T(n) \leq ((c + d)k + c) + ((c + d)(n - k - 1) + c) + d = (c + d)n + c$.

Attraversamento

Pre-ordine e Post-ordine

- Non hanno un significato preciso come l'attraversamento simmetrico, che restituisce la lista ordinata.
- Per esercizio, scrivere sia la versione ricorsiva sia quella iterativa di entrambi.

- Gli ABR supportano SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR.
- Ciascuna richiede un tempo $O(h)$ dove h è l'altezza dell'ABR in input.
- Vediamole in sintesi. Per i dettagli si veda [CLRS].

Interrogazione di un ABR

SEARCH

Algoritmo: SEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

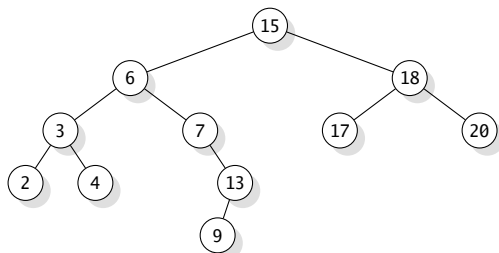
Output: Un nodo che contiene k se esiste, null altrimenti

```
1: if  $x == \text{null}$  or  $k == x.\text{key}$  then return  $x$ 
2: if  $k < x.\text{key}$  then return SEARCH( $x.\text{left}, k$ )
3: return SEARCH( $x.\text{right}, k$ )
```

- I nodi incontrati durante la ricerca formano un cammino verso il basso dell'albero.
- Il tempo di esecuzione è quindi $O(h)$, dove h è l'altezza dell'albero.

Interrogazione di un ABR

SEARCH



- La ricerca di 13 porta a eseguire il percorso $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$.
- Quindi 3 chiamate ricorsive.

Interrogazione di un ABR

SEARCH iterativa

Algoritmo: ITERSEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

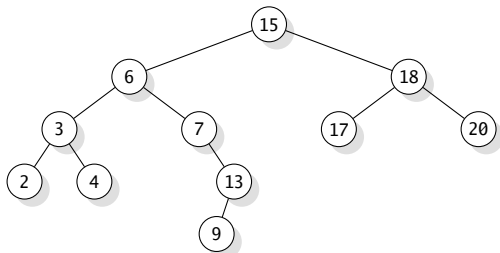
Output: Un nodo che contiene k se esiste, null altrimenti

```
1: while  $x \neq \text{null}$  and  $k \neq x.\text{key}$  do  
2:    $x = (k < x.\text{key}) ? x.\text{left} : x.\text{right}$   
3: return  $x$ 
```

- La versione iterativa è più efficiente a livello pratico (cambia solo la costante nascosta in $O(h)$). **Perché?**

Interrogazione di un ABR

MINIMUM, MAXIMUM



- L'elemento con chiave **minima** è il nodo **più a sx** nell'albero (per la proprietà fondante degli ABR).
 - Si scende l'albero fino a quando il figlio sx diventa null: il nodo contiene il minimo.
- Analogamente, l'elemento con chiave **massima** è il nodo **più a dx**.
- Entrambe le ricerche hanno costo computazionale in tempo $O(h)$, h = altezza dell'albero.

Interrogazione di un ABR

MINIMUM, MAXIMUM

Algoritmo: MINIMUM(x)

Input: Un nodo x di un ABR

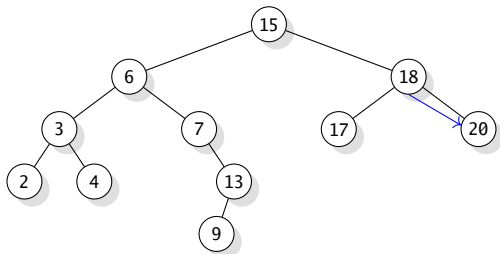
Output: Un nodo che contiene la chiave minima dell'ABR

```
1: while  $x.left \neq null$  do  
2:    $x = x.left$   
3: return  $x$ 
```

La procedura MAXIMUM è lasciata per esercizio.

Interrogazione di un ABR

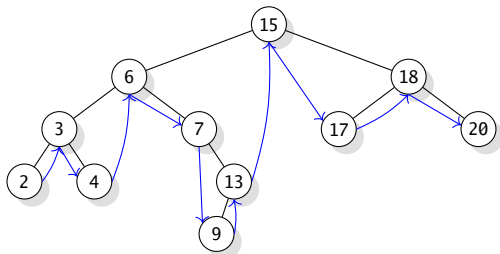
SUCCESSOR, PREDECESSOR



- Data una chiave k , talvolta è importante trovare il successore (il successore di k è la chiave che compare immediatamente dopo nella visita in ordine).
- Se tutte le chiavi sono distinte, il successore di k è la chiave più piccola k' tale che $k < k'$.
- In un ABR il successore e il predecessore si possono trovare senza confrontare le chiavi.
- Nell'esempio, il successore di 4 è 6 e di 13 è 15!

Interrogazione di un ABR

SUCCESSOR, PREDECESSOR



- Data una chiave k , talvolta è importante trovare il successore (il successore di k è la chiave che compare immediatamente dopo nella visita in ordine).
- Se tutte le chiavi sono distinte, il successore di k è la chiave più piccola k' tale che $k < k'$.
- In un ABR il successore e il predecessore si possono trovare senza confrontare le chiavi.
- Gli archi rappresentano la relazione successore.

Interrogazione di un ABR

SUCCESSOR, PREDECESSOR

- Se un nodo x ha il sottoalbero dx , allora il successore è il MINIMUM del sottoalbero dx .
- Altrimenti, il successore è il primo nodo y per il quale x appartiene al sottoalbero sx di y .

Algoritmo: SUCCESSOR(x)

Input: Un nodo x di un ABR

Output: Il nodo successore di x nella sequenza ordinata dell'ABR

```
1: if  $x.right \neq null$  then return MINIMUM( $x.right$ )  
2:  $y := x.p$   
3: while  $y \neq null$  and  $x == y.right$  do  
4:    $x = y$   
5:    $y = y.p$   
6: return  $y$ 
```

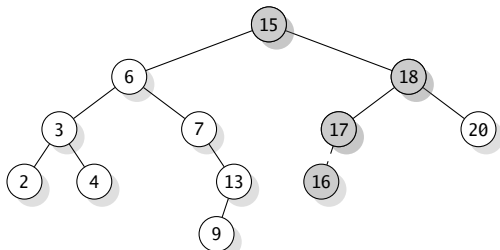
La procedura PREDECESSOR è lasciata per esercizio.

Inserimento e cancellazione in un ABR

- Inserire o cancellare un elemento modifica la struttura dell'albero.
- Si deve quindi garantire che la nuova struttura soddisfi ancora la proprietà fondante (ripristino).
- L'inserimento è semplice, mentre la cancellazione richiede un ripristino un po' più complicato.

Inserimento in un ABR

- Si vuole inserire un nodo z (chiave $z.key$ e figli NULL) in un ABR T .
- La procedura INSERT deve modificare T e z in modo che z diventi un nodo di T nella giusta posizione.
- INSERT non controlla se l'elemento è già presente (quindi può inserire più copie dello stesso valore)
- INSERT inserisce sempre una foglia!



Esempio d'inserimento del nodo con valore 16.

I nodi con sfondo grigio indicano il percorso fatto per trovare la posizione corretta per il nodo.

Inserimento in un ABR

Algoritmo: INSERT(t, z)

Input: Un ABR t e un nodo $z \notin t$

```
1:  $y = null$ 
2:  $x = t.root$ 
3: while  $x \neq null$  do
4:    $y = x$ 
5:    $x = (z.key < x.key) ? x.left : x.right$ 
6:  $z.p = y$ 
7: if  $y == null$  then  $t.root = z$  // t era vuoto
8: else
9:   if  $z.key < y.key$  then
10:     $y.left = z$ 
11:   else
12:     $y.right = z$ 
```

Cancellazione in un ABR

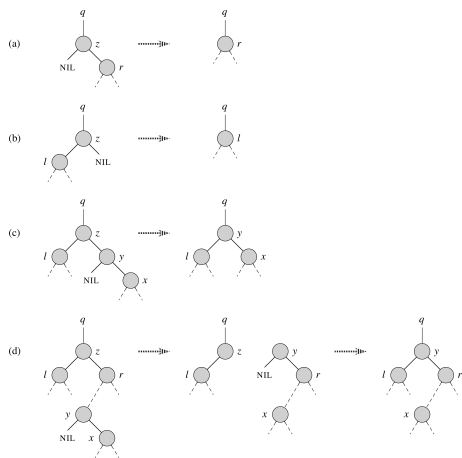
Si consideri che si debba cancellare il nodo z .

I casi possibili sono:

- ① **z non ha figli** $\Rightarrow$ Si cancella z .
- ② **manca uno dei figli di z** $\Rightarrow$ Il figlio presente prende il posto di z .
- ③ **ci sono entrambi i figli di z** $\Rightarrow$ Si cerca il successore, y , di z .
 y viene messo al posto di z .
 - y è nel sottoalbero dx.
 - Se y è foglia, sostituzione è semplice.
- ④ Altrimenti si devono gestire sia i figli di z sia il figlio dx di y per spostare y al posto di z .
Richiede qualche spostamento di nodi (caso più complicato).

Cancellazione in un ABR

I casi ② e ④ vengono gestiti come i seguenti 4 casi particolari: (a) e (b) come casi di ② e (c) e (d) come casi particolari di ④:



Cancellazione in un ABR

Soluzione CLRS

Procedura per spostare sotto-alberi all'interno di un ABR.

Algoritmo: TRANSPLANT(t, u, v)

Input: Un ABR t e due nodi u, v di t . v può essere null.

Output: In t , sostituisce il sottoalbero con radice u con il sottoalbero con radice v .

1: **if** $u.p == \text{null}$ **then**

2: $t.\text{root} = v$

3: **else**

4: **if** $u == u.p.\text{left}$ **then**

// u è figlio sx

5: $u.p.\text{left} = v$

6: **else**

7: $u.p.\text{right} = v$

8: **if** $v \neq \text{null}$ **then** $v.p = u.p$

// Inserisci v in t

TRANSPLANT ha complessità in tempo $O(1)$.

Cancellazione in un ABR

Soluzione CLRS

Algoritmo: DELETE(t, z)

Input: Un ABR t e un nodo $z \in t$

Output: In t , cancella z mantenendo t ABR

```
1: if  $z.left == null$  then TRANSPLANT( $t, z, z.right$ ); return // (a)
2: if  $z.right == null$  then TRANSPLANT( $t, z, z.left$ ); return // (b)
3:  $y = \text{MINIMUM}(z.right)$ 
4: if  $y.p \neq z$  then // (d)
5:   TRANSPLANT( $t, y, y.right$ ) // (d) al posto di  $y$ , suo figlio  $x$ 
6:    $y.right = z.right$  // (d)  $r$  diventa figlio dx di  $y$ 
7:    $y.right.p = y$ 
8: TRANSPLANT( $t, z, y$ ) // ultimo passo di (d) e (c)
9:  $y.left = z.left$ 
10:  $y.left.p = y$ 
```

DELETE ha complessità in tempo $O(h)$ perché può eseguire MINIMUM.

- La complessità delle operazioni fondamentali degli ABR è $O(h)$, dove h è l'altezza dell'albero.
- Dalle proprietà degli alberi binari, $\lfloor \lg n \rfloor \leq h \leq n - 1$ dove n è il numero dei nodi.
- Il caso peggiore è quando un ABR è una lista.
 - è sufficiente costruire un ABR inserendo i valori uno a uno da una sequenza ordinata.
- Non sono note formule che danno una stima dell'altezza media di un ABR dopo una serie di operazioni d'inserimento e/o cancellazione.

- Un risultato importante è dato solo sull'altezza media di un albero quando viene costruito in modo casuale.
- Dato un insieme di chiavi, un ABR è costruito in modo casuale quando, a partire dall'albero vuoto, si inseriscono le chiavi una alla volta scegliendole casualmente dall'insieme.
- In tal caso, vale il seguente teorema (senza dimostrazione).

Teorema 2

*L'altezza media di un ABR costruito in modo casuale con n chiavi **distinte** è $O(\lg n)$.*

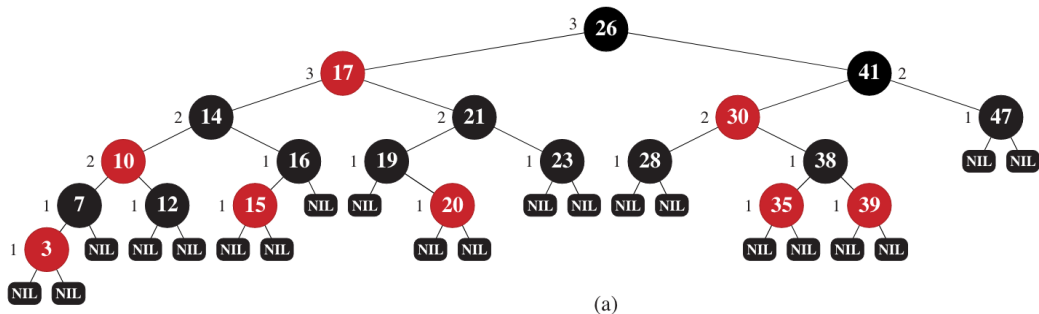
Superamento degli ABR

ABR rosso-neri

- Un ABR permette di eseguire le operazioni di ricerca in modo ottimale quando è **bilanciato** (altezza $h \approx \lg n$).
- Il problema principale è che operazioni di inserimento e cancellazione possono sbilanciare un albero fino a farlo diventare degenerare (lista).
- Esistono varianti degli ABR che permettono di mantenere il più possibile il bilanciamento dell'albero.
- La variante più conosciuta è l'**ABR rosso-nero**:
 - ogni nodo binario ha un colore: rosso o nero.
 - la radice è nera, come ogni riferimento a null;
 - se un nodo è rosso, i figli sono neri (se sono null, sono neri per punto precedente)
 - per ogni nodo, tutti i cammini semplici che vanno dal nodo alle foglie contengono lo stesso numero di nodi neri.

Superamento degli ABR

ABR rosso-neri



Lemma 1

L'altezza di un ABR rosso-nero con n nodi è al più $2 \lg(n + 1)$.

In questa lezione non si presentano gli algoritmi d'inserimento e cancellazione di un ABR rosso-nero.

Alberi binari di ricerca

Radix tree

Idea

Un **radix tree** (o **radix trie**, **trie compresso**) è una struttura ad albero usata per rappresentare un insieme di stringhe binarie sfruttando i **prefissi comuni**.

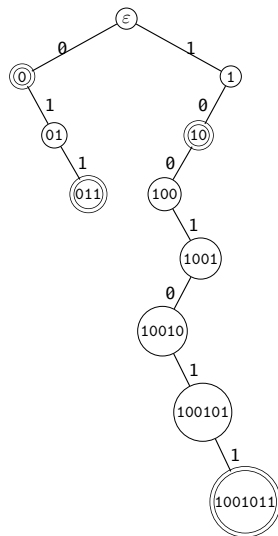
- Ogni arco è etichettato con un simbolo dell'alfabeto, nel nostro caso 0 oppure 1.
- Il cammino dalla radice a un nodo rappresenta il prefisso ottenuto concatenando le etichette degli archi attraversati.
- Una stringa s appartiene all'insieme se esiste un nodo marcato come **terminale** il cui cammino dalla radice è esattamente s .
- Stringhe con prefisso comune condividono la parte iniziale del cammino.
- Operazioni come ricerca, inserimento e visita richiedono un tempo proporzionale alla lunghezza della stringa considerata.

Alberi binari di ricerca

Radix tree: esempio

Consideriamo $S = \{0, 011, 10, 1001011\}$.

- Ogni arco è etichettato con 0 oppure 1.
- I nodi con doppio bordo sono **terminali**.
- Il cammino dalla radice a un nodo terminale rappresenta una stringa dell'insieme.
- I prefissi comuni sono condivisi:
 - 0 e 011;
 - 10 e 1001011.



In questa lezione si sono introdotti i seguenti concetti:

- la definizione di albero binario di ricerca (ABR),
- i metodi di attraversamento di ABR,
- la ricerca in ABR,
- l'inserimento e la cancellazione di nodi in ABR,
- cenno agli ABR rosso-neri,
- cenno ai radix tree.

- Studiare il capitolo 12.
- Dimostrare correttezza algoritmo **inOrderWalk** per induzione.
Suggerimento: la base è un nodo (banale). Quindi si suppone che valga per alberi di dimensione $n - 1$ e si considera un albero di n nodi. Un albero di n nodi può essere visto come un nodo con due sottoalberi, uno di $0 \leq k \leq n - 1$ nodi, e l'altro di $n - 1 - k$ nodi.
- Risolvere gli esercizi 12.1-2, 12.1-3, 12.1-4, 12.2-2, 12.2-3, 12.2-4, 12.2-5, 12.3-1, 12.3-3 [CLRS].

Suggerimento: nell'esercizio 12.1-3, il testo richiede di pensare anche a una soluzione che non usi lo stack. La soluzione non è banale perché chiede in pratica di trasformare l'albero in albero threaded (con fili). Una pagina che spiega questa soluzione è <https://www.techiedelight.com/inorder-tree-traversal-iterative-recursive>